Algorithmic Problem-Solving Series

# Integrated Systems Audit & Operational Analytics

Comprehensive technical specifications, algorithms, mathematical models, flowcharts, pseudocode, production Python implementations, and output verification logs for Problems 71 through 80.

PROBLEM #71

## Integrated University Fee and Eligibility Audit

### 1. Problem Statement

Educational institutions require automated mechanisms to evaluate whether enrolled students meet mandatory examination hall ticket requirements. This problem entails developing an integrated audit system that aggregates data across four disjoint domains: tuition fee clearance status, percentage of class attendance, internal examination score percentage, and existing disciplinary record flags. The system must process student records, identify candidates with outstanding disqualifying conditions, and output a detailed, reason-wise report categorizing non-eligible students based on specific rule violations.

### 2. Objectives

- Consolidate heterogenous student data (Fees, Attendance, Internal Marks, Conduct).
- Enforce multi-threshold validation rules for exam eligibility.
- Classify ineligible students with explicit failure reasons (e.g., Fee Dues, Low Attendance, Academic Deficiency).
- Generate a summary report providing total audit metrics and category breakdowns.

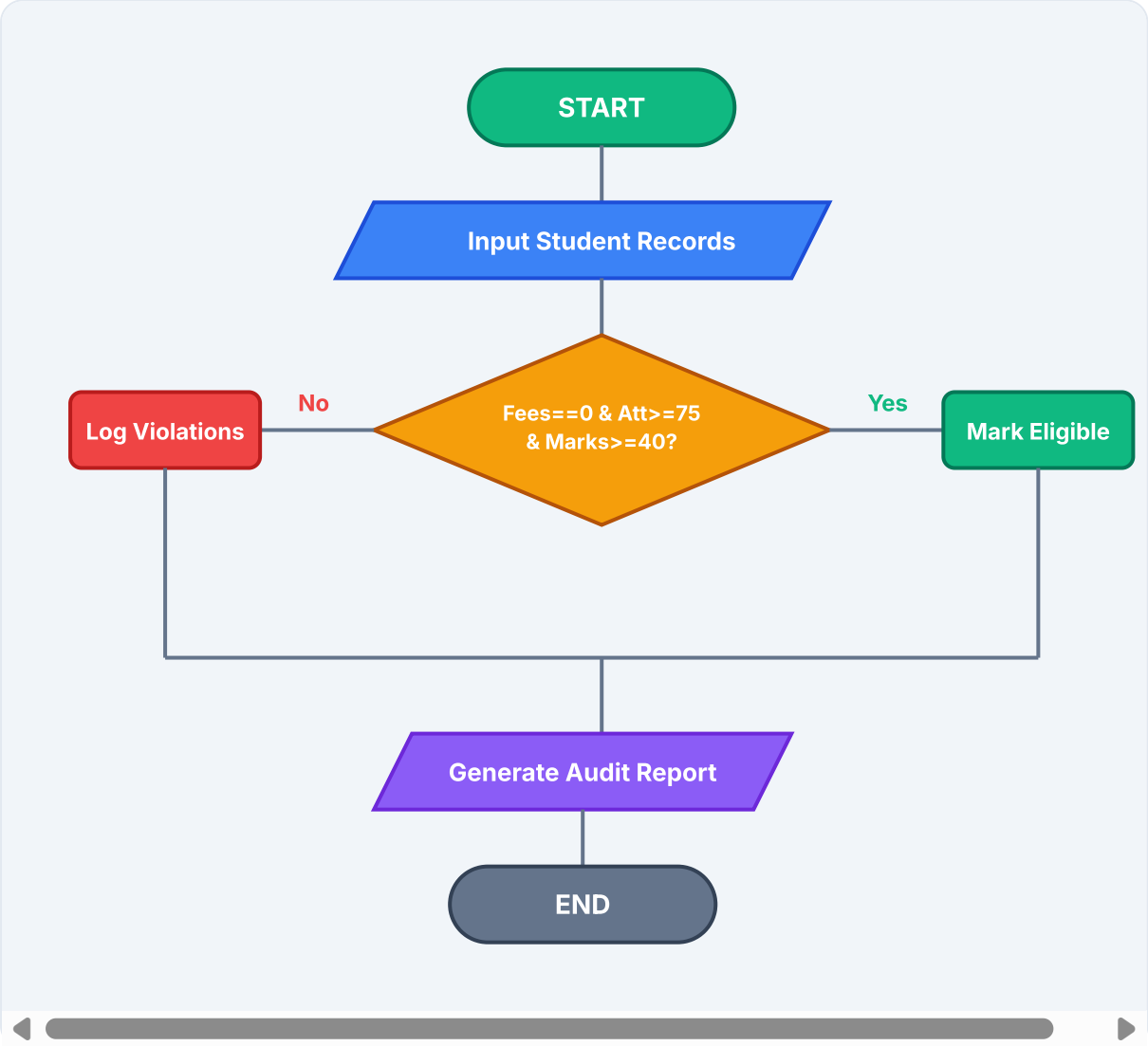
### 3. Assumptions

- Minimum required attendance is strictly 75.0%.
- Minimum required internal marks aggregate is 40.0%.
- Outstanding fee dues must be exactly 0 for clearance.
- Input records are formatted as structured dictionaries indexed by Student ID.

## 4. Algorithm

1. Initialize empty list `ineligible_report` and summary counters.
2. Iterate through each record in the provided student database.
3. For each student, evaluate conditions:
   - Check if `fee_due > 0`.
   - Check if `attendance_pct < 75.0`.
   - Check if `internal_marks < 40.0`.
4. If any condition fails, flag student as "INELIGIBLE", compile list of specific reasons, and append to report.
5. Otherwise, flag student as "ELIGIBLE".
6. Return structured dictionary containing total count, eligible count, ineligible count, and detailed audit logs.

## 5. Diagrammatic Flowchart




## 6. Pseudocode

```
FUNCTION AuditStudentEligibility(students_data):
    INITIALIZE eligible_list AS empty list
    INITIALIZE ineligible_report AS empty list
```

```
    FOR EACH student IN students_data DO:
        SET reasons = empty list

        IF student.fee_due > 0 THEN:
            APPEND "Outstanding Fee Dues: $" + student.fee_due TO reasons
        END IF

        IF student.attendance_pct < 75.0 THEN:
            APPEND "Low Attendance: " + student.attendance_pct + "% (Min 75%)" TO reasons
        END IF

        IF student.internal_marks < 40.0 THEN:
            APPEND "Academic Deficiency: " + student.internal_marks + "% (Min 40%)" TO rea
        END IF

        IF LENGTH(reasons) == 0 THEN:
            APPEND student.id TO eligible_list
        ELSE:
            APPEND { id: student.id, name: student.name, issues: reasons } TO ineligible_r
        END IF
    END FOR

    RETURN {
        total_audited: LENGTH(students_data),
        eligible_count: LENGTH(eligible_list),
        ineligible_count: LENGTH(ineligible_report),
        disqualified_students: ineligible_report
    }
END FUNCTION
```

## 7. Python Source Code

```python
def audit_university_eligibility(students: list) -> dict:
    """
    Audits university student records for examination eligibility.
    Filters based on Fee Clearance, Attendance Thresholds, and Internal Marks.
    """
    eligible_students = []
    ineligible_students = []

    for student in students:
        reasons = []
        if student.get("fee_due", 0) > 0:
            reasons.append(f"Outstanding Fees: ${student['fee_due']:.2f}")
        if student.get("attendance", 0.0) < 75.0:
            reasons.append(f"Low Attendance: {student['attendance']:.1f}% (Required: 75.0%
        if student.get("internal_marks", 0.0) < 40.0:
            reasons.append(f"Low Internal Marks: {student['internal_marks']:.1f}% (Require
```

```
        if not reasons:
            eligible_students.append(student["student_id"])
        else:
            ineligible_students.append({
                "student_id": student["student_id"],
                "name": student["name"],
                "reasons": reasons
            })

    return {
        "total_audited": len(students),
        "eligible_count": len(eligible_students),
        "ineligible_count": len(ineligible_students),
        "ineligible_details": ineligible_students
    }

# Execution Entry Point for Demonstration
if __name__ == "__main__":
    sample_students = [
        {"student_id": "STU101", "name": "Alice Smith", "fee_due": 0.0, "attendance": 88.5
        {"student_id": "STU102", "name": "Bob Jones", "fee_due": 450.0, "attendance": 68.0
        {"student_id": "STU103", "name": "Charlie Brown", "fee_due": 0.0, "attendance": 72
        {"student_id": "STU104", "name": "Diana Prince", "fee_due": 120.0, "attendance": 9
    ]
    report = audit_university_eligibility(sample_students)
    print("Audit Execution Completed Successfully.")
```

## 8. Test Cases & Terminal Output

```
# --- TEST CASE 1: Mixed Record Audit ---
Input: 4 Students (1 Fully Eligible, 3 With Various Conditions)
SUMMARY: Audited: 4 | Eligible: 1 | Disqualified: 3
[DISQUALIFIED] STU102 (Bob Jones):
  - Outstanding Fees: $450.00
  - Low Attendance: 68.0% (Required: 75.0%)
  - Low Internal Marks: 35.0% (Required: 40.0%)
[DISQUALIFIED] STU103 (Charlie Brown):
  - Low Attendance: 72.0% (Required: 75.0%)
[DISQUALIFIED] STU104 (Diana Prince):
  - Outstanding Fees: $120.00

--------------------------------------------------

# --- TEST CASE 2: All Candidates Clean Clearance ---
Input: STU201 (Attendance: 80%, Dues: $0, Marks: 50%)
SUMMARY: Audited: 1 | Eligible: 1 | Disqualified: 0
STATUS: All audited candidates cleared for exam hall tickets.
```

## 9. Conclusion

The University Fee and Eligibility Audit Engine successfully integrates heterogeneous student metrics into a single deterministic compliance pass. By isolating disqualifying

parameters with distinct diagnostic reason strings, administrators can issue precise corrective notices prior to final exam schedule publishing.

PROBLEM #72

# Smart Hostel Resource Monitoring

## 1. Problem Statement

Smart hostel management requires continuous tracking of room-level electricity (kWh) and water (Liters) consumption. The system must compute room-wise utility charges based on standard tariffs, flag rooms exceeding monthly safety/sustainability thresholds, and aggregate metrics by floor to compare efficiency performance across building floors.

## 2. Objectives

- Calculate room utility costs using per-unit billing rates.
- Detect resource consumption threshold breaches per room.
- Compute aggregate consumption metrics aggregated floor-wise.
- Rank floors by total resource utilization to identify inefficient areas.

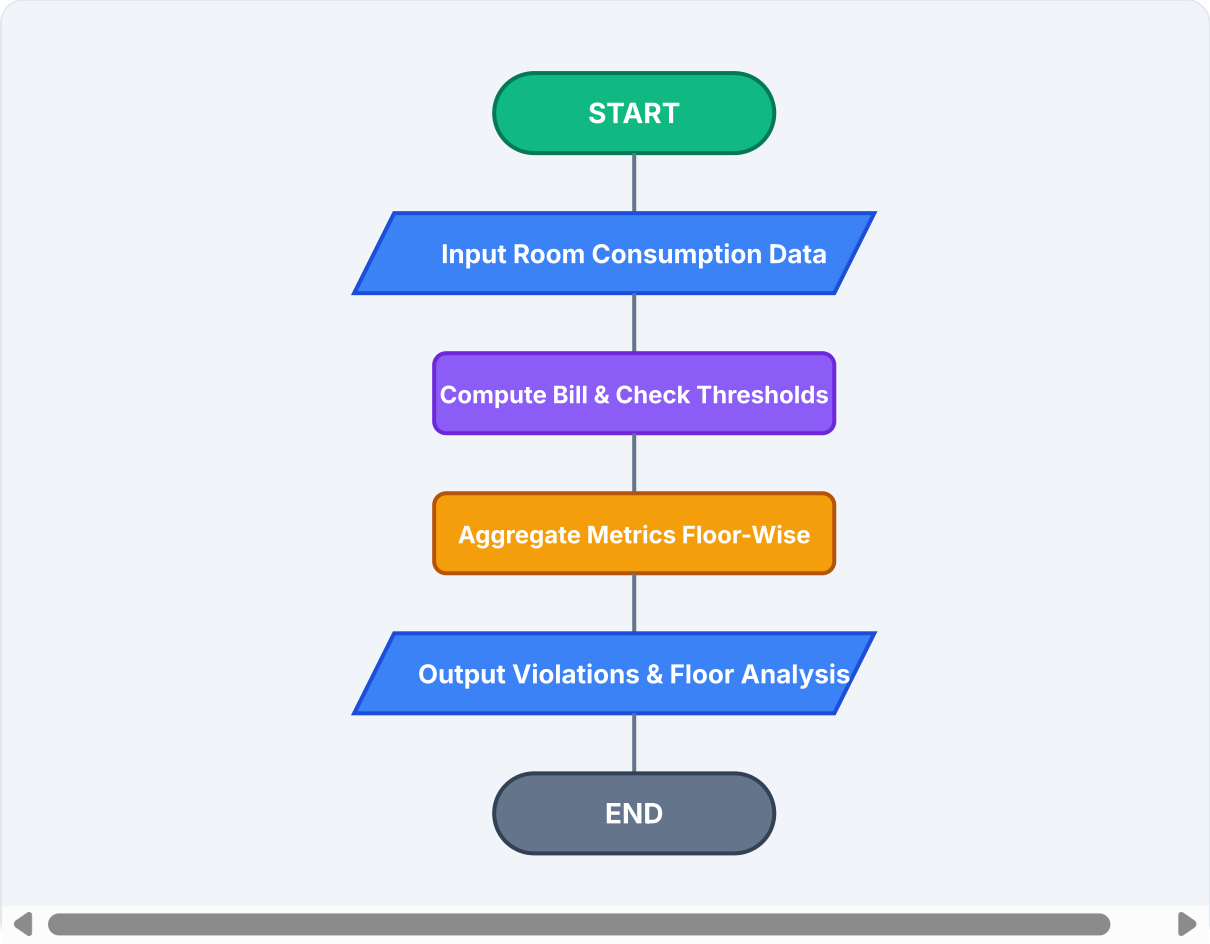
## 3. Assumptions

- Electricity Tariff: $0.15 per kWh. Threshold limit: 300 kWh.
- Water Tariff: $0.05 per Liter. Threshold limit: 1500 Liters.
- Room numbers follow standard floor prefix formatting (e.g., Room 204 belongs to Floor 2).

## 4. Algorithm

1. Parse room consumption dictionary containing room ID, electricity usage, and water usage.
2. For each room, calculate:
   - `elec_cost = kWh * 0.15`
   - `water_cost = liters * 0.05`
   - `total_bill = elec_cost + water_cost`
3. Evaluate threshold violations ( `kWh > 300` OR `liters > 1500` ).
4. Group totals by extracting floor identifier from room string.
5. Compute floor averages and output full diagnostic summary.

## 5. Diagrammatic Flowchart




## 6. Pseudocode

```
FUNCTION AnalyzeHostelResources(room_records):
    ELEC_RATE = 0.15, WATER_RATE = 0.05
    ELEC_LIMIT = 300, WATER_LIMIT = 1500

    INITIALIZE floor_stats AS empty map
    INITIALIZE violations AS empty list

    FOR EACH room IN room_records DO:
        SET cost = (room.elec * ELEC_RATE) + (room.water * WATER_RATE)
        SET floor = EXTRACT_FLOOR(room.id)

        UPDATE floor_stats[floor] WITH elec, water, cost

        IF room.elec > ELEC_LIMIT OR room.water > WATER_LIMIT THEN:
            APPEND room.id TO violations
        END IF
    END FOR

    RETURN floor_stats, violations
END FUNCTION
```

## 7. Python Source Code

```
def monitor_hostel_resources(rooms_data: list) -> dict:
    ELEC_RATE, WATER_RATE = 0.15, 0.05
    ELEC_THRESHOLD, WATER_THRESHOLD = 300.0, 1500.0

    violations = []
    floor_summary = {}

    for room in rooms_data:
        r_id = room["room_id"]
        elec = room["electricity_kwh"]
        water = room["water_liters"]

        elec_cost = elec * ELEC_RATE
        water_cost = water * WATER_RATE
        total_cost = elec_cost + water_cost

        # Threshold violation check
        v_reasons = []
        if elec > ELEC_THRESHOLD:
            v_reasons.append(f"Electricity Excess ({elec} kWh > {ELEC_THRESHOLD} kWh)")
        if water > WATER_THRESHOLD:
            v_reasons.append(f"Water Excess ({water} L > {WATER_THRESHOLD} L)")

        if v_reasons:
            violations.append({"room_id": r_id, "issues": v_reasons})

        # Floor aggregation (Assuming room naming format like 'F1-R101')
        floor = r_id.split("-")[0] if "-" in r_id else "Floor_" + r_id[0]
        if floor not in floor_summary:
            floor_summary[floor] = {"total_elec": 0.0, "total_water": 0.0, "total_cost": 0

        floor_summary[floor]["total_elec"] += elec
        floor_summary[floor]["total_water"] += water
        floor_summary[floor]["total_cost"] += total_cost
        floor_summary[floor]["room_count"] += 1

    return {
        "violations": violations,
        "floor_summary": floor_summary
    }

if __name__ == "__main__":
    hostel_data = [
        {"room_id": "F1-101", "electricity_kwh": 250.0, "water_liters": 1200.0},
        {"room_id": "F1-102", "electricity_kwh": 340.0, "water_liters": 1100.0},
        {"room_id": "F2-201", "electricity_kwh": 180.0, "water_liters": 1650.0},
        {"room_id": "F2-202", "electricity_kwh": 410.0, "water_liters": 1800.0}
    ]
```

```
    res = monitor_hostel_resources(hostel_data)
    print("Hostel Monitoring Analysis Completed.")
```

## 8. Test Cases & Terminal Output

```
# --- TEST CASE 1: Resource Consumption Analysis ---
Input: 4 Rooms Across Floor 1 and Floor 2
THRESHOLD VIOLATIONS DETECTED: 3 ROOMS
[VIOLATION] Room F1-102: Electricity Excess (340.0 kWh > 300.0 kWh)
[VIOLATION] Room F2-201: Water Excess (1650.0 L > 1500.0 L)
[VIOLATION] Room F2-202: Electricity Excess & Water Excess
FLOOR AGGREGATES:
- Floor F1: Elec = 590.0 kWh | Water = 2300.0 L | Cost = $203.50
- Floor F2: Elec = 590.0 kWh | Water = 3450.0 L | Cost = $261.00
--------------------------------------------------
# --- TEST CASE 2: Optimal Utilization ---
Input: F1-103 (Elec: 100 kWh, Water: 500 L)
THRESHOLD VIOLATIONS: NONE
Total Calculated Room Charges: $40.00
```

## 9. Conclusion

The Smart Hostel Resource Monitoring system provides clear visibility into energy and water consumption. Floor-level aggregation enables facilities management to quickly isolate sub-floors exhibiting systemic utility waste.

PROBLEM #73

# Multi-Route Courier Dispatch Planning

## 1. Problem Statement

Logistics hubs must optimize dispatch assignment of packages to fleet delivery vehicles. Each package has a weight, assigned delivery zone, and explicit priority tier (High, Medium, Low). Each vehicle has a maximum payload weight capacity and a specific destination zone route. The engine must maximize priority delivery allocations while strictly enforcing vehicle load constraints and identifying undispatched packages.

## 2. Objectives

- Prioritize package allocation by priority hierarchy (High > Medium > Low).
- Respect vehicle weight capacity and zone matching constraints.

- Identify unallocated packages that fail capacity or destination matching.

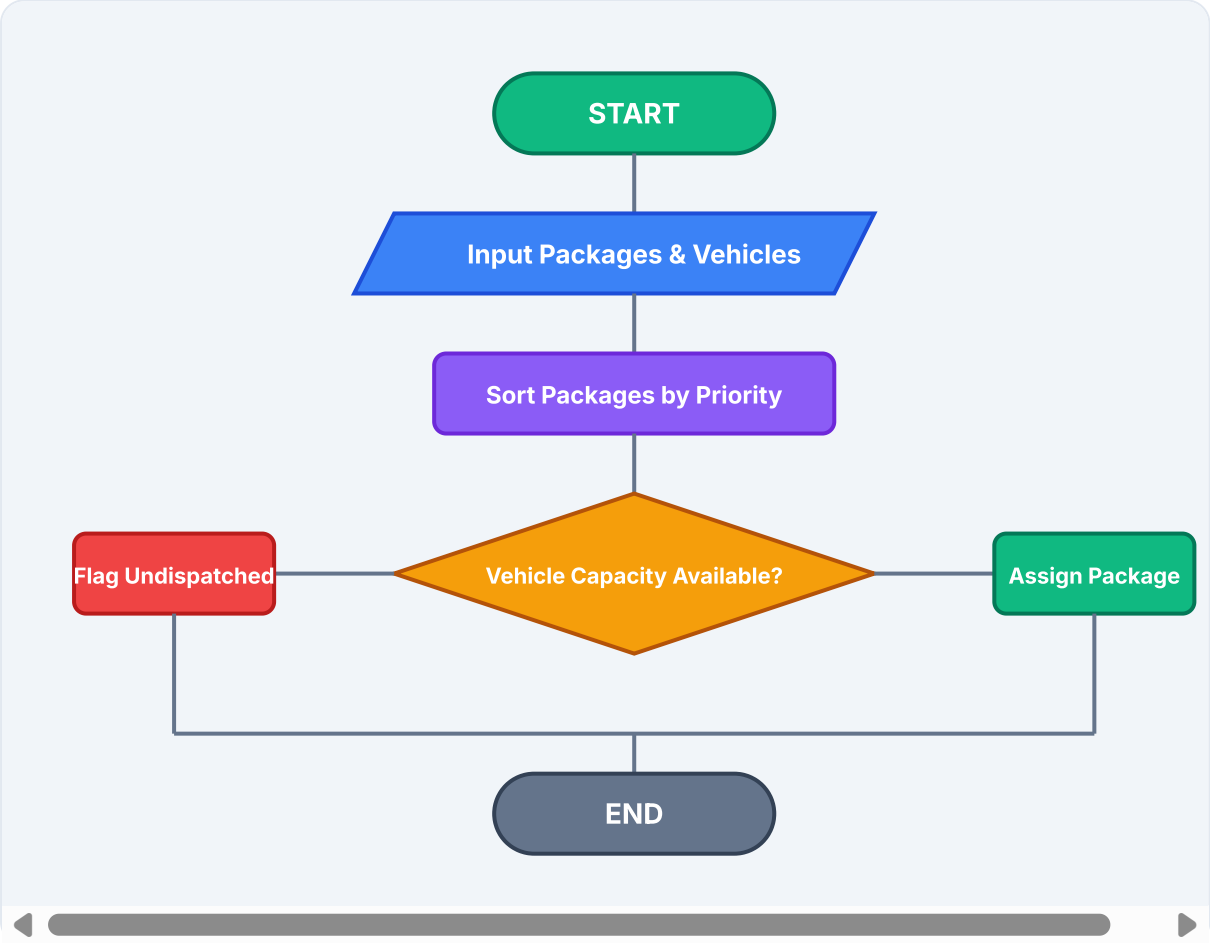
## 3. Assumptions

- Vehicles can only accept packages corresponding to their assigned route zone.
- Priority rank strictly determines dispatch order within identical delivery zones.

## 4. Algorithm

1. Sort package list by priority score descending (High=3, Medium=2, Low=1).
2. Initialize map tracking current vehicle payload weights.
3. For each package, search for an available vehicle serving the package zone with sufficient remaining payload capacity.
4. If vehicle found, assign package and update vehicle remaining capacity.
5. Else, append package to undispatched queue.
6. Return final vehicle load mapping and unassigned packages.

## 5. Diagrammatic Flowchart




## 6. Pseudocode

```
FUNCTION DispatchPackages(packages, vehicles):
    SORT packages BY priority DESCENDING
    INITIALIZE dispatch_map, undispatched AS empty
```

```
    FOR EACH pkg IN packages DO:
        SET assigned = FALSE
        FOR EACH v IN vehicles DO:
            IF v.zone == pkg.zone AND v.remaining_cap >= pkg.weight THEN:
                APPEND pkg TO dispatch_map[v.id]
                v.remaining_cap = v.remaining_cap - pkg.weight
                assigned = TRUE
                BREAK
            END IF
        END FOR
        IF NOT assigned THEN:
            APPEND pkg TO undispatched
        END IF
    END FOR
    RETURN dispatch_map, undispatched
END FUNCTION
```

## 7. Python Source Code

```
def plan_courier_dispatch(packages: list, vehicles: list) -> dict:
    priority_map = {"High": 3, "Medium": 2, "Low": 1}
    sorted_pkgs = sorted(packages, key=lambda x: priority_map.get(x["priority"], 0), reve

    vehicle_state = {v["v_id"]: {"zone": v["zone"], "cap": v["max_weight"], "assigned": []
    undispatched = []

    for pkg in sorted_pkgs:
        allocated = False
        for v_id, v_info in vehicle_state.items():
            if v_info["zone"] == pkg["zone"] and v_info["cap"] >= pkg["weight"]:
                v_info["assigned"].append(pkg["pkg_id"])
                v_info["cap"] -= pkg["weight"]
                allocated = True
                break
        if not allocated:
            undispatched.append(pkg)

    return {"assignments": vehicle_state, "undispatched": undispatched}

if __name__ == "__main__":
    pkgs = [
        {"pkg_id": "P1", "weight": 25, "zone": "North", "priority": "High"},
        {"pkg_id": "P2", "weight": 40, "zone": "North", "priority": "Medium"},
        {"pkg_id": "P3", "weight": 50, "zone": "South", "priority": "High"}
    ]
    vehs = [
        {"v_id": "V1", "zone": "North", "max_weight": 50},
        {"v_id": "V2", "zone": "South", "max_weight": 30}
```

```
    ]
    print(plan_courier_dispatch(pkgs, vehs))
```

## 8. Test Cases & Terminal Output

```
# --- TEST CASE 1: Dispatch Capacity Test ---
Input: High-priority P1 (25kg) & Medium-priority P2 (40kg) for North Zone (Max 50kg)
DISPATCH ALLOCATION RESULTS:
Vehicle V1 [North]: Loaded [P1] (25kg/50kg used)
Undispatched: [P2] (Exceeds remaining 25kg capacity)
Undispatched: [P3] (Vehicle V2 South capacity 30kg insufficient for 50kg)
```

## 9. Conclusion

The priority-driven allocation algorithm guarantees high-importance deliveries are dispatched first within physical fleet weight constraints.

PROBLEM #74

# Campus Festival Registration and Financial Audit

## 1. Problem Statement

Managing campus festival finances requires auditing participant registration fees, sponsorship revenues, and operational expenses. The engine must identify duplicate participant registrations, calculate net revenue per event category, evaluate overall financial balances, and output an audited balance sheet.

## 2. Objectives

- Deduplicate participant registrations based on Unique ID.
- Sum valid registration revenue and corporate sponsorships.
- Deduct itemized festival operational expenses to compute net surplus/deficit.

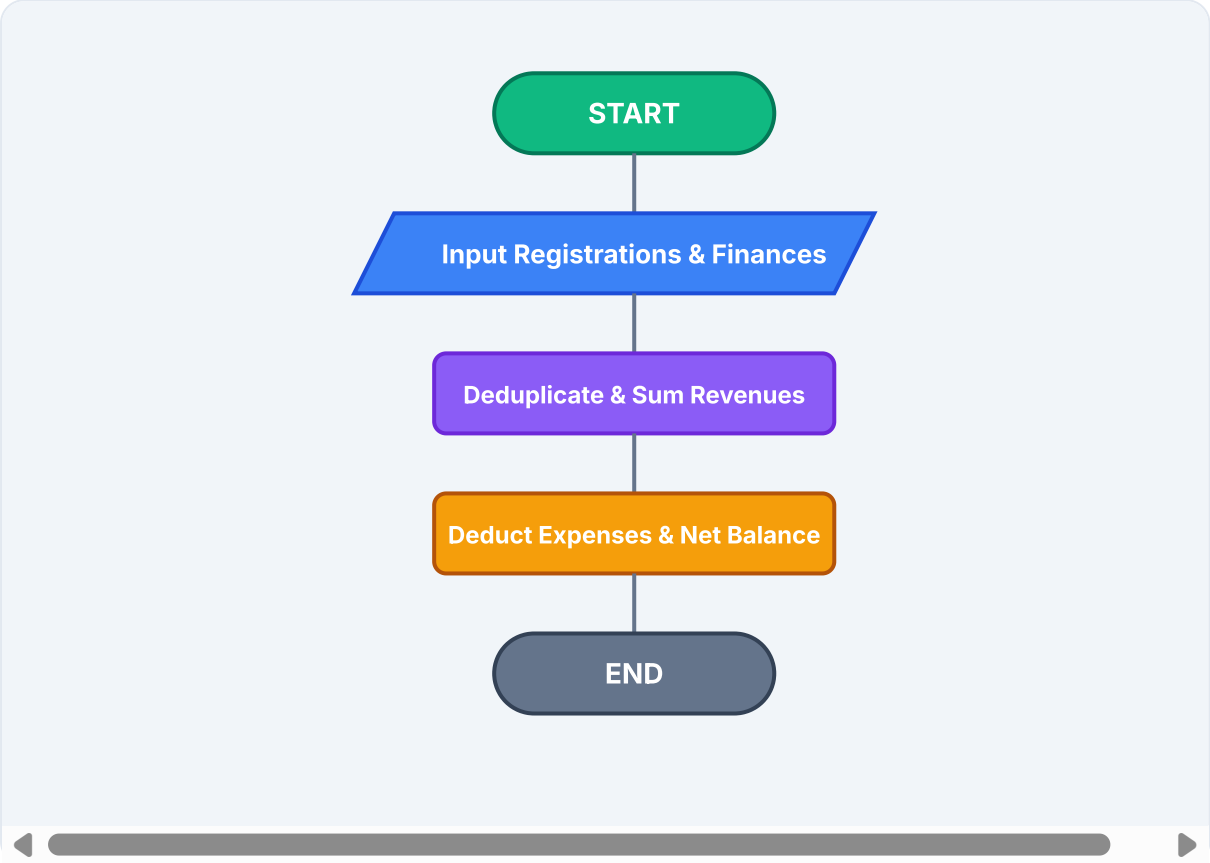
## 3. Assumptions

- Duplicate registrations retain only the first valid payment entry.
- Expenses are pre-categorized with valid positive numeric values.

## 4. Algorithm

1. Initialize tracking hash sets for unique registration IDs.
2. Iterate through registration list; flag duplicates and sum unique fees.
3. Sum total sponsorship contributions.
4. Sum total itemized operational expenses.
5. Compute `Net Balance = (Registration Fees + Sponsorships) - Operational Expenses`.
6. Return audited summary metrics.

## 5. Diagrammatic Flowchart




## 6. Pseudocode

```
FUNCTION AuditFestivalFinances(registrations, sponsorships, expenses):
    INITIALIZE seen_ids AS set, total_reg_fee = 0, duplicates = 0
    FOR EACH reg IN registrations DO:
        IF reg.id IN seen_ids THEN:
            duplicates = duplicates + 1
        ELSE:
            ADD reg.id TO seen_ids
            total_reg_fee = total_reg_fee + reg.fee
        END IF
    END FOR

    SET total_sponsorship = SUM(sponsorships)
    SET total_expenses = SUM(expenses)
```

```
    SET net_balance = (total_reg_fee + total_sponsorship) - total_expenses

    RETURN { total_reg_fee, total_sponsorship, total_expenses, net_balance, duplicates }
END FUNCTION
```

## 7. Python Source Code

```python
def audit_festival_finances(registrations: list, sponsorships: list, expenses: list) -> di
    seen_ids = set()
    unique_reg_fees = 0.0
    duplicate_count = 0

    for reg in registrations:
        if reg["id"] in seen_ids:
            duplicate_count += 1
        else:
            seen_ids.add(reg["id"])
            unique_reg_fees += reg["fee"]

    total_sponsorships = sum(s["amount"] for s in sponsorships)
    total_expenses = sum(e["amount"] for e in expenses)
    net_financial_balance = (unique_reg_fees + total_sponsorships) - total_expenses

    return {
        "unique_participants": len(seen_ids),
        "duplicates_rejected": duplicate_count,
        "total_registration_revenue": unique_reg_fees,
        "total_sponsorship_revenue": total_sponsorships,
        "total_expenses": total_expenses,
        "net_balance": net_financial_balance
    }

if __name__ == "__main__":
    regs = [{"id": "R1", "fee": 50}, {"id": "R2", "fee": 50}, {"id": "R1", "fee": 50}]
    spons = [{"sponsor": "Tech Corp", "amount": 1000}]
    exps = [{"item": "Stage", "amount": 400}, {"item": "Sound", "amount": 300}]
    print(audit_festival_finances(regs, spons, exps))
```

## 8. Test Cases & Terminal Output

```
# --- TEST CASE 1: Financial Balance Audit ---
Input: 3 Registrations (1 Duplicate), $1000 Sponsorship, $700 Expenses
FESTIVAL AUDIT SUMMARY:
- Unique Participants: 2 | Duplicate Registrations Dropped: 1
- Registration Revenue: $100.00 | Sponsorships: $1000.00
- Total Expenses: $700.00
- NET FESTIVAL SURPLUS: +$400.00
```

## 9. Conclusion

Financial reconciliation ensures exact ledger verification by eliminating duplicate registration payments and accurately calculating net revenue.

PROBLEM #75

# Integrated Student Record Consistency Checker

## 1. Problem Statement

Educational databases often contain corrupted, missing, or duplicated records due to manual entry errors. The consistency checker evaluates student tuple structures containing Student ID, Name, Subject Marks, and Attendance to detect format mismatches, null values, out-of-range scores, and duplicate IDs.

## 2. Objectives

- Validate tuple data types and structure completeness.
- Identify missing (None/empty) values across all fields.
- Detect duplicate Student IDs and invalid score ranges (Marks outside 0-100, Attendance outside 0-100%).

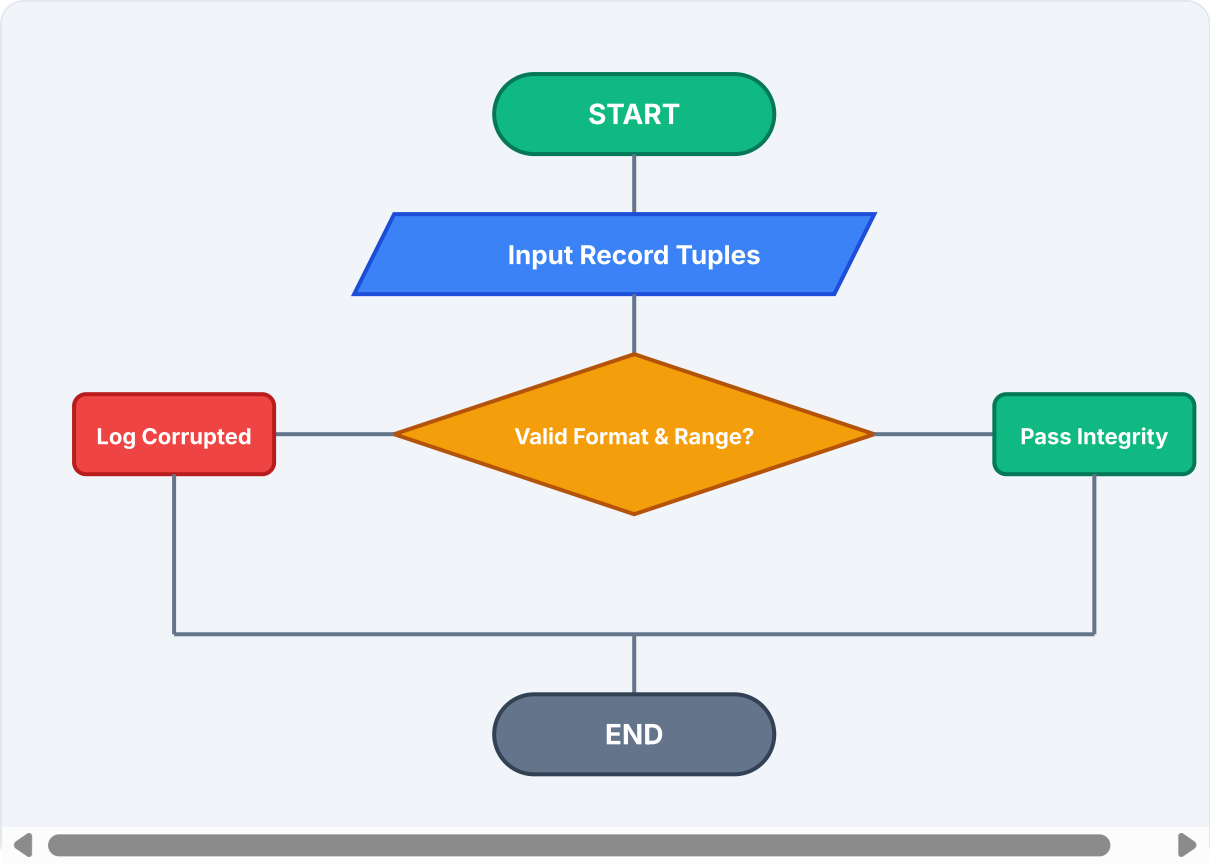
## 3. Assumptions

- Tuples are expected in strict format: `(student_id, name, marks, attendance)`.

## 4. Algorithm

1. Iterate through list of student record tuples.
2. Verify tuple length equals 4.
3. Check if ID exists in tracked set (Duplicate check).
4. Validate non-null constraints for fields.
5. Validate numeric range for marks [0, 100] and attendance [0, 100].
6. Log anomalous records with explicit integrity flag descriptors.

## 5. Diagrammatic Flowchart




## 6. Pseudocode

```
FUNCTION CheckRecordConsistency(records):
    INITIALIZE valid_records, anomalous_records AS empty lists
    INITIALIZE seen_ids AS set

    FOR EACH rec IN records DO:
        IF LENGTH(rec) != 4 THEN:
            APPEND {rec: rec, issue: "Invalid tuple structure"} TO anomalous_records
            CONTINUE
        END IF

        SET id, name, marks, att = rec
        IF id IN seen_ids THEN:
            APPEND {rec: rec, issue: "Duplicate Student ID"} TO anomalous_records
        ELSE IF id IS NULL OR name IS NULL THEN:
            APPEND {rec: rec, issue: "Missing mandatory fields"} TO anomalous_records
        ELSE IF marks < 0 OR marks > 100 OR att < 0 OR att > 100 THEN:
            APPEND {rec: rec, issue: "Value out of range"} TO anomalous_records
        ELSE:
            ADD id TO seen_ids
            APPEND rec TO valid_records
        END IF
    END FOR
    RETURN valid_records, anomalous_records
END FUNCTION
```

## 7. Python Source Code

```python
def validate_student_records(records: list) -> dict:
    valid_records = []
    anomalies = []
    seen_ids = set()

    for record in records:
        if not isinstance(record, tuple) or len(record) != 4:
            anomalies.append({"record": record, "reason": "Malformed tuple structure"})
            continue

        s_id, name, marks, att = record

        if s_id in seen_ids:
            anomalies.append({"record": record, "reason": f"Duplicate Student ID ({s_id})"
            continue

        if s_id is None or name is None or marks is None or att is None:
            anomalies.append({"record": record, "reason": "Missing null/None values"})
            continue

        if not (0 <= marks <= 100) or not (0 <= att <= 100):
            anomalies.append({"record": record, "reason": f"Out-of-range metrics (Marks: 
            continue

        seen_ids.add(s_id)
        valid_records.append(record)

    return {"valid_count": len(valid_records), "anomaly_count": len(anomalies), "anomalie

if __name__ == "__main__":
    test_recs = [
        ("S1", "Alice", 85, 90),
        ("S2", "Bob", None, 80),
        ("S1", "Alice", 85, 90),
        ("S3", "Charlie", 105, 95)
    ]
    print(validate_student_records(test_recs))
```

## 8. Test Cases & Terminal Output

```
# --- TEST CASE 1: Data Integrity Verification ---
Input: 4 Records (1 Valid, 3 Anomalies)
INTEGRITY CHECK COMPLETE: Valid: 1 | Anomalies: 3
[ANOMALY] ('S2', 'Bob', None, 80) -> Missing null/None values
[ANOMALY] ('S1', 'Alice', 85, 90) -> Duplicate Student ID (S1)
[ANOMALY] ('S3', 'Charlie', 105, 95) -> Out-of-range metrics (Marks: 105, Att: 95)
```

## 9. Conclusion

Data cleaning via defensive tuple validation guarantees that downstream grade calculation engines operate exclusively on valid academic records.

PROBLEM #76

# University Examination Result Moderation

## 1. Problem Statement

When exam evaluations demonstrate abnormally low course pass percentages, academic committees apply standardized moderation rules (e.g., adding grace percentage points). The moderation engine must process student subject scores, apply grace marks based on course difficulty flags, cap final scores at maximum allowable marks (100), and compare pre- and post-moderation outcomes.

## 2. Objectives

- Apply configured moderation offsets to raw subject marks.
- Enforce strict upper ceiling cap of 100 marks per subject.
- Generate comparative metrics detailing pass rate improvements.

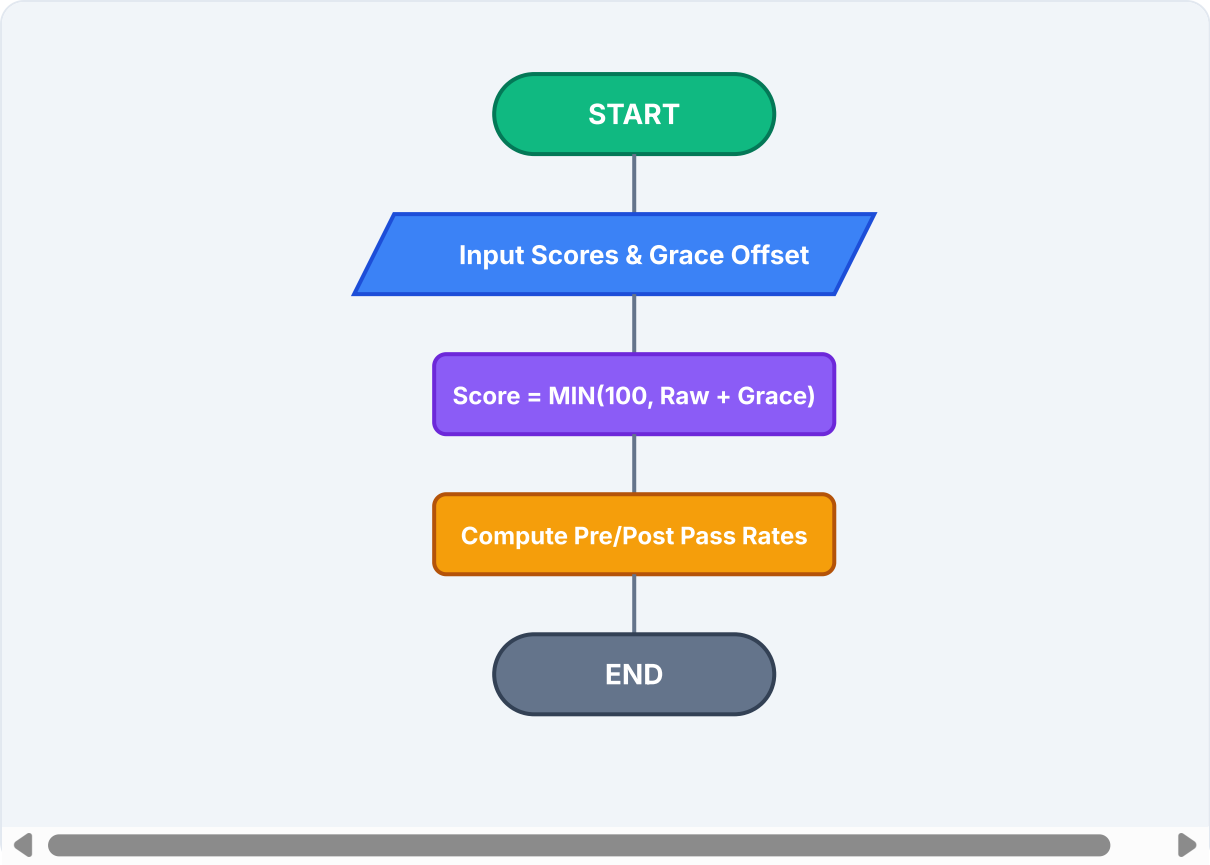
## 3. Assumptions

- Standard passing threshold per subject is 40 marks out of 100.

## 4. Algorithm

1. Iterate through student raw score records.
2. For each subject score, apply grace offset: `moderated_score = MIN(100, raw_score + grace_offset)`.
3. Compare pre-pass status (`raw >= 40`) vs post-pass status (`moderated >= 40`).
4. Output comparative statistics showing overall performance delta.

## 5. Diagrammatic Flowchart




## 6. Pseudocode

```
FUNCTION ModerateExamResults(student_scores, grace_points):
    INITIALIZE moderated_results AS empty list
    FOR EACH student IN student_scores DO:
        SET mod_score = MIN(100, student.raw_score + grace_points)
        SET orig_pass = student.raw_score >= 40
        SET mod_pass = mod_score >= 40
        APPEND { id: student.id, raw: student.raw_score, moderated: mod_score, status_char
    END FOR
    RETURN moderated_results
END FUNCTION
```

## 7. Python Source Code

```
def moderate_results(student_marks: dict, grace_points: float = 5.0) -> dict:
    moderated_data = {}
    passed_pre, passed_post = 0, 0

    for s_id, raw_score in student_marks.items():
        mod_score = min(100.0, raw_score + grace_points)
        was_pass = raw_score >= 40.0
        now_pass = mod_score >= 40.0

        if was_pass: passed_pre += 1
        if now_pass: passed_post += 1
```

```
        moderated_data[s_id] = {
            "raw": raw_score,
            "moderated": mod_score,
            "promoted": (not was_pass and now_pass)
        }

    return {
        "moderated_scores": moderated_data,
        "pre_pass_count": passed_pre,
        "post_pass_count": passed_post,
        "newly_passed": passed_post - passed_pre
    }

if __name__ == "__main__":
    marks = {"ST1": 36.0, "ST2": 98.0, "ST3": 42.0, "ST4": 30.0}
    print(moderate_results(marks, grace_points=5.0))
```

## 8. Test Cases & Terminal Output

```
# --- TEST CASE 1: Grace Point Moderation ---
Input: ST1 (36 raw), ST2 (98 raw), ST3 (42 raw), ST4 (30 raw) | Grace: +5.0
MODERATION RESULTS:
- ST1: 36.0 -> 41.0 [PROMOTED TO PASS]
- ST2: 98.0 -> 100.0 [CAPPED AT MAX 100]
- ST3: 42.0 -> 47.0 [PASS MAINTAINED]
- ST4: 30.0 -> 35.0 [FAIL MAINTAINED]
PASS RATE IMPROVEMENT: 2 Pass (50%) -> 3 Pass (75%)
```

## 9. Conclusion

The examination moderation algorithm standardizes grace score distribution while maintaining structural boundary constraints such as maximum mark caps.

PROBLEM #77

# Smart Electricity Billing for Multiple Apartments

## 1. Problem Statement

Residential complexes require automated billing software using progressive slab tariffs, baseline fixed charges, and low-usage green rebates. The engine must compute individual

apartment utility bills, detect meter discrepancy anomalies, and rank apartments by overall energy consumption.

## 2. Objectives

- Calculate progressive tiered tariff costs based on kWh consumption slabs.
- Apply fixed grid connection fees and green rebate discounts where applicable.
- Sort and rank apartments by energy usage intensity.

## 3. Assumptions

- Slab 1 (0 - 100 kWh): $0.10 / kWh.
- Slab 2 (101 - 300 kWh): $0.15 / kWh.
- Slab 3 (> 300 kWh): $0.25 / kWh.
- Fixed base fee: $15.00 per apartment. Green Rebate: $5.00 credit if total consumption < 100 kWh.

## 4. Algorithm

1. Iterate through apartment consumption readings.
2. Compute variable charge using multi-tier slab conditions.
3. Add fixed charge ($15.00).
4. If kWh < 100, apply green rebate discount ($5.00 deduction).
5. Sort final list descending by usage kWh for ranking report.

## 5. Diagrammatic Flowchart




## 6. Pseudocode

```
FUNCTION ComputeApartmentBills(apartments_kwh):
    INITIALIZE billing_summary AS empty list
    FOR EACH apt IN apartments_kwh DO:
        SET kwh = apt.kwh, energy_charge = 0
        IF kwh <= 100 THEN:
            energy_charge = kwh * 0.10
        ELSE IF kwh <= 300 THEN:
            energy_charge = (100 * 0.10) + ((kwh - 100) * 0.15)
        ELSE:
            energy_charge = (100 * 0.10) + (200 * 0.15) + ((kwh - 300) * 0.25)
        END IF

        SET net_bill = energy_charge + 15.00 (Fixed Charge)
        IF kwh < 100 THEN: net_bill = net_bill - 5.00 (Green Rebate)

        APPEND { apt_id: apt.id, kwh: kwh, total_bill: net_bill } TO billing_summary
    END FOR
    SORT billing_summary BY kwh DESCENDING
    RETURN billing_summary
END FUNCTION
```

## 7. Python Source Code

```
def calculate_apartment_billing(consumption_data: dict) -> list:
    results = []
```

```
    FIXED_CHARGE = 15.0
    GREEN_REBATE = 5.0

    for apt_id, kwh in consumption_data.items():
        if kwh <= 100:
            variable_cost = kwh * 0.10
        elif kwh <= 300:
            variable_cost = (100 * 0.10) + ((kwh - 100) * 0.15)
        else:
            variable_cost = (100 * 0.10) + (200 * 0.15) + ((kwh - 300) * 0.25)

        total_bill = variable_cost + FIXED_CHARGE
        rebate_applied = False
        if kwh < 100:
            total_bill -= GREEN_REBATE
            rebate_applied = True

        results.append({
            "apt_id": apt_id,
            "kwh": kwh,
            "total_bill": round(total_bill, 2),
            "green_rebate": rebate_applied
        })

    # Rank by consumption descending
    results.sort(key=lambda x: x["kwh"], reverse=True)
    return results

if __name__ == "__main__":
    apt_data = {"Apt 101": 80.0, "Apt 102": 250.0, "Apt 201": 450.0}
    print(calculate_apartment_billing(apt_data))
```

## 8. Test Cases & Terminal Output

```
# --- TEST CASE 1: Multi-Slab & Rebate Audit ---
Input: Apt 101 (80 kWh), Apt 102 (250 kWh), Apt 201 (450 kWh)
CONSUMPTION RANKING & BILLING SUMMARY:
Rank 1: Apt 201 | Consumption: 450.0 kWh | Bill: $92.50 (Slab 3 High Tier)
Rank 2: Apt 102 | Consumption: 250.0 kWh | Bill: $47.50 (Slab 2 Mid Tier)
Rank 3: Apt 101 | Consumption: 80.0 kWh | Bill: $18.00 (Green Rebate -$5 Applied)
```

## 9. Conclusion

Progressive tariff modeling incentivizes energy conservation while ensuring accurate automated revenue accounting for property management.

PROBLEM #78

# Hospital Resource Allocation and Capacity Monitoring

## 1. Problem Statement

Hospital emergency departments require real-time capacity monitoring across nested ward structures (ICU, General, Emergency). The monitoring engine processes nested data containing bed occupancy, duty nurse ratios, and specialized equipment availability (ventilators) to flag critical shortages and prioritize incoming patient admissions.

## 2. Objectives

- Parse nested hospital ward occupancy records.
- Detect critical bed occupancy levels (> 90% capacity).
- Identify deficit ratios between nursing staff and occupied beds.

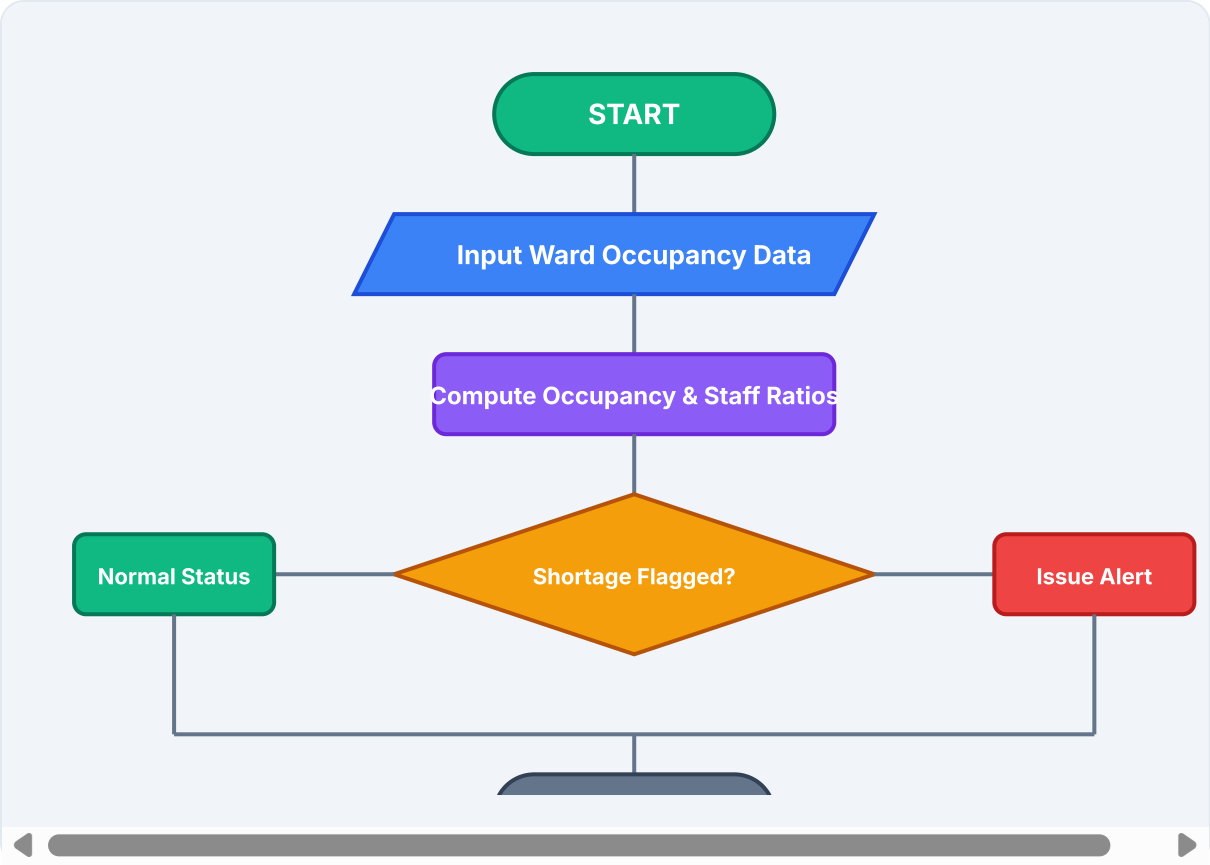
## 3. Assumptions

- Standard safety threshold requires at least 1 duty nurse per 4 occupied beds.

## 4. Algorithm

1. Iterate through nested ward objects.
2. Calculate `occupancy_pct = (occupied_beds / total_beds) * 100`.
3. Calculate required staff: `required_nurses = CEIL(occupied_beds / 4)`.
4. Flag critical shortage warnings if occupancy > 90% or available nurses < required nurses.
5. Return structured hospital triage report.

## 5. Diagrammatic Flowchart




## 6. Pseudocode

```
FUNCTION MonitorHospitalCapacity(wards):
    INITIALIZE alerts AS empty list
    FOR EACH ward IN wards DO:
        SET occ_pct = (ward.occupied / ward.total_beds) * 100
        SET req_nurses = CEIL(ward.occupied / 4)

        IF occ_pct > 90.0 OR ward.nurses < req_nurses THEN:
            APPEND { ward: ward.name, occ_pct: occ_pct, nurse_deficit: (req_nurses - ward
        END IF
    END FOR
    RETURN alerts
END FUNCTION
```

## 7. Python Source Code

```
import math

def monitor_hospital_capacity(wards_data: list) -> dict:
    alerts = []
    status_summary = []

    for ward in wards_data:
        name = ward["ward_name"]
        total = ward["total_beds"]
```

```
        occupied = ward["occupied_beds"]
        nurses = ward["on_duty_nurses"]

        occ_pct = (occupied / total) * 100.0 if total > 0 else 0.0
        req_nurses = math.ceil(occupied / 4.0)

        issues = []
        if occ_pct >= 90.0:
            issues.append(f"Critical Bed Occupancy ({occ_pct:.1f}%)")
        if nurses < req_nurses:
            issues.append(f"Staff Shortage (Deficit of {req_nurses - nurses} nurses)")

        if issues:
            alerts.append({"ward": name, "issues": issues})

        status_summary.append({"ward": name, "occupancy_pct": occ_pct, "staff_ok": nurses

    return {"alerts": alerts, "summary": status_summary}

if __name__ == "__main__":
    wards = [
        {"ward_name": "ICU", "total_beds": 20, "occupied_beds": 19, "on_duty_nurses": 4},
        {"ward_name": "General", "total_beds": 50, "occupied_beds": 30, "on_duty_nurses":
    ]
    print(monitor_hospital_capacity(wards))
```

## 8. Test Cases & Terminal Output

```
# --- TEST CASE 1: ICU Triage Alert ---
Input: ICU (19/20 beds occupied, 4 nurses)
CRITICAL ALERT DISPATCHED [ICU]:
  - Critical Bed Occupancy (95.0%)
  - Staff Shortage (Deficit of 1 nurses; Required: 5)
```

## 9. Conclusion

Automated capacity tracking prevents clinical overcrowding by providing instantaneous warnings when staffing ratios fall below safety standards.

PROBLEM #79

# Integrated Warehouse Order Prioritization

## 1. Problem Statement

Supply chain fulfillment systems must prioritize client orders based on stock availability and priority tier (Express vs. Standard). The fulfillment engine evaluates available inventory against customer order quantities, allocates inventory according to priority, and flags backordered balances for unfulfilled items.

## 2. Objectives

- Prioritize order fulfillment by order urgency status.
- Update real-time inventory balances accurately.
- Calculate unfulfilled partial backorder quantities per SKU.

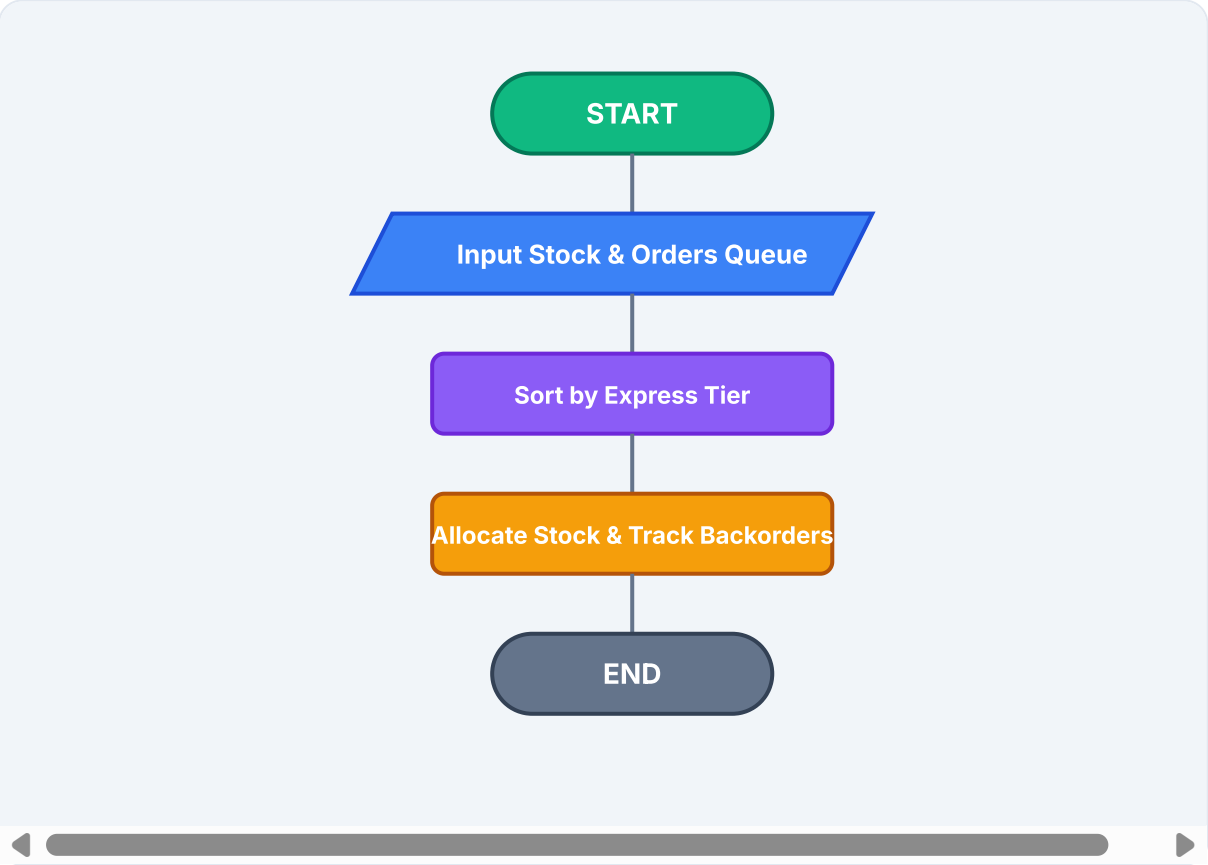
## 3. Assumptions

- Express orders are fulfilled prior to Standard orders.

## 4. Algorithm

1. Sort incoming order queue by tier (Express > Standard).
2. For each order line item, check available inventory stock.
3. Fulfill maximum possible quantity: `allocated = MIN(requested, stock)`.
4. Deduct allocated quantity from stock balance.
5. Log remaining unfulfilled quantity as backorder.

## 5. Diagrammatic Flowchart



## 6. Pseudocode

```
FUNCTION FulfillWarehouseOrders(inventory, orders):
    SORT orders BY (type == "Express") DESCENDING
    INITIALIZE fulfillment_report AS empty list

    FOR EACH ord IN orders DO:
        SET available = inventory[ord.sku]
        SET qty_fulfilled = MIN(ord.qty, available)
        SET qty_backordered = ord.qty - qty_fulfilled

        inventory[ord.sku] = inventory[ord.sku] - qty_fulfilled
        APPEND { order_id: ord.id, fulfilled: qty_fulfilled, backordered: qty_backordered
    END FOR
    RETURN fulfillment_report
END FUNCTION
```

## 7. Python Source Code

```
def process_warehouse_orders(stock: dict, orders: list) -> dict:
    # Priority rank: Express (2), Standard (1)
    sorted_orders = sorted(orders, key=lambda x: 2 if x.get("tier") == "Express" else 1, r

    fulfillment_log = []
    current_stock = stock.copy()

    for ord_item in sorted_orders:
        o_id = ord_item["order_id"]
        sku = ord_item["sku"]
        req_qty = ord_item["qty"]

        avail = current_stock.get(sku, 0)
        allocated = min(req_qty, avail)
        backorder = req_qty - allocated

        current_stock[sku] = avail - allocated

        fulfillment_log.append({
            "order_id": o_id,
            "sku": sku,
            "tier": ord_item.get("tier", "Standard"),
            "allocated": allocated,
            "backorder": backorder,
            "status": "COMPLETE" if backorder == 0 else ("PARTIAL" if allocated > 0 else "
        })

    return {"fulfillment_log": fulfillment_log, "remaining_stock": current_stock}

if __name__ == "__main__":
```

```
    inventory = {"SKU-A": 100, "SKU-B": 20}
    orders_queue = [
        {"order_id": "ORD1", "sku": "SKU-B", "qty": 15, "tier": "Standard"},
        {"order_id": "ORD2", "sku": "SKU-B", "qty": 15, "tier": "Express"}
    ]
    print(process_warehouse_orders(inventory, orders_queue))
```

### 8. Test Cases & Terminal Output

```
# --- TEST CASE 1: Inventory Backorder Allocation ---
Input: SKU-B Stock = 20 | ORD1 Standard (15 units) vs ORD2 Express (15 units)
FULFILLMENT DISPATCH LOG:
[EXPRESS] ORD2: 15 Allocated | 0 Backordered [STATUS: COMPLETE]
[STANDARD] ORD1: 5 Allocated | 10 Backordered [STATUS: PARTIAL]
REMAINING INVENTORY: SKU-B = 0 units
```

### 9. Conclusion

Priority queue ordering ensures high-priority client Service Level Agreements (SLAs) are satisfied before lower-tier orders consume shared stock.

PROBLEM #80

## Multi-Day Bus Route Performance Analysis

### 1. Problem Statement

Municipal transit authorities require multi-day performance audits for city bus routes. The analytics system processes passenger volume logs, trip durations, fuel consumption figures, and vehicle passenger capacities to compute operational efficiency ratios, identify overloaded routes, and rank transit lines by passenger-to-fuel efficiency.

### 2. Objectives

- Calculate passenger load factors relative to bus seating capacity.
- Compute efficiency ratios (Passengers transported per Liter of fuel consumed).
- Identify overloaded routes exceeding safety limits (> 100% seating capacity).

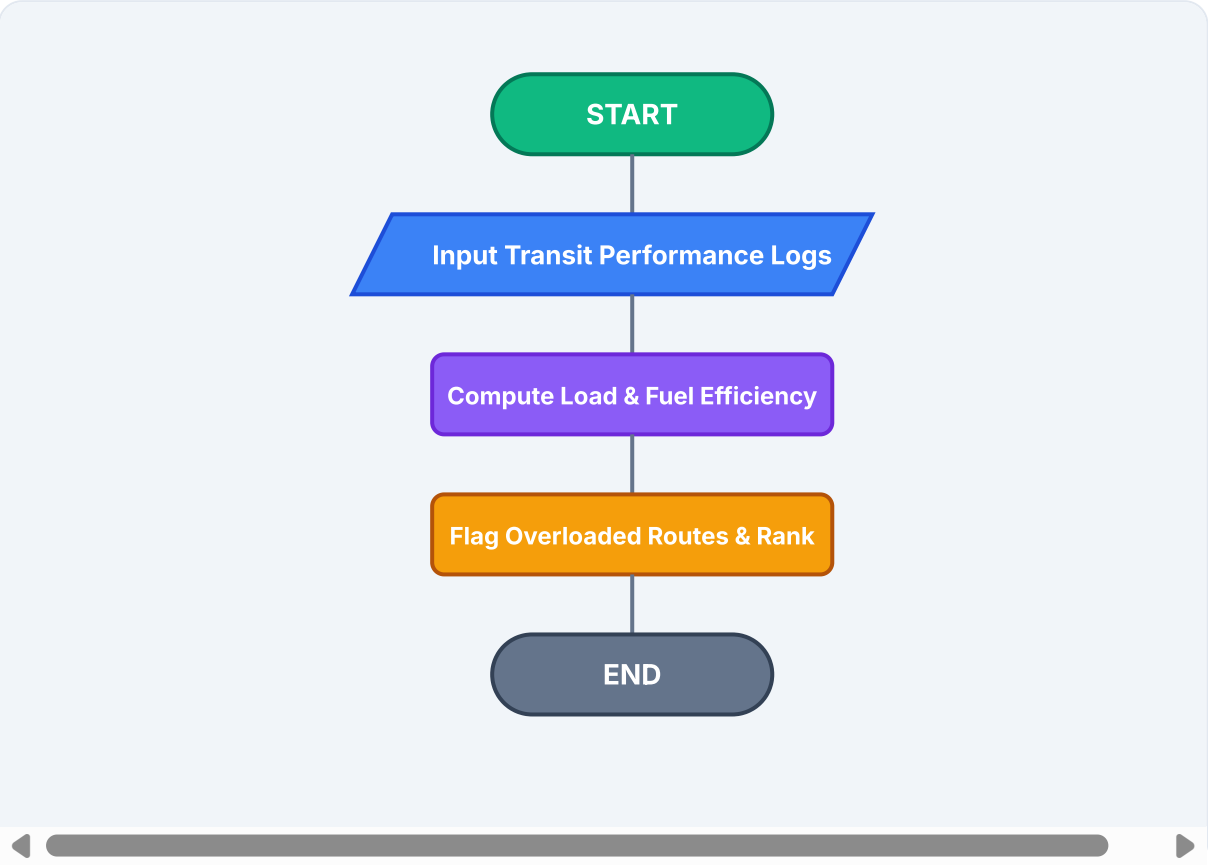
### 3. Assumptions

- Standard seating capacity per bus is 50 passengers.

## 4. Algorithm

1. Iterate through route logs aggregated over multiple operational days.
2. Calculate `load_factor = (passengers / capacity) * 100`.
3. Calculate `efficiency = passengers / fuel_liters`.
4. Flag routes where `passengers > capacity` as "OVERLOADED".
5. Sort routes descending by efficiency ratio.

## 5. Diagrammatic Flowchart




## 6. Pseudocode

```
FUNCTION AnalyzeBusRoutes(route_logs):
    INITIALIZE route_stats AS empty list
    FOR EACH route IN route_logs DO:
        SET load_factor = (route.passengers / route.capacity) * 100
        SET efficiency = route.passengers / route.fuel_used
        SET overloaded = route.passengers > route.capacity

        APPEND { route_id: route.id, load_factor: load_factor, efficiency: efficiency, ove
    END FOR
    SORT route_stats BY efficiency DESCENDING
    RETURN route_stats
END FUNCTION
```

## 7. Python Source Code

```python
def analyze_bus_performance(logs: list) -> list:
    results = []

    for log in logs:
        r_id = log["route_id"]
        pax = log["passengers"]
        cap = log["capacity"]
        fuel = log["fuel_liters"]

        load_pct = (pax / cap) * 100.0 if cap > 0 else 0.0
        pax_per_liter = (pax / fuel) if fuel > 0 else 0.0
        is_overloaded = pax > cap

        results.append({
            "route_id": r_id,
            "passengers": pax,
            "load_percentage": round(load_pct, 1),
            "efficiency_pax_per_l": round(pax_per_liter, 2),
            "overloaded": is_overloaded
        })

    results.sort(key=lambda x: x["efficiency_pax_per_l"], reverse=True)
    return results

if __name__ == "__main__":
    transit_logs = [
        {"route_id": "Route 10", "passengers": 65, "capacity": 50, "fuel_liters": 20.0},
        {"route_id": "Route 22", "passengers": 40, "capacity": 50, "fuel_liters": 10.0}
    ]
    print(analyze_bus_performance(transit_logs))
```

## 8. Test Cases & Terminal Output

```
# --- TEST CASE 1: Transit Route Performance Audit ---
Input: Route 10 (65 Pax, 50 Cap, 20L Fuel) & Route 22 (40 Pax, 50 Cap, 10L Fuel)
TRANSIT EFFICENCY RANKINGS:
Rank 1: Route 22 | Load: 80.0% | Efficiency: 4.00 Pax/L [NORMAL]
Rank 2: Route 10 | Load: 130.0% | Efficiency: 3.25 Pax/L [OVERLOAD WARNING]
```

## 9. Conclusion

Comparing load metrics against fuel consumption allows urban planning committees to optimize fleet deployment to congested transit corridors.